

Superscalar Architecture

Week 5: Going Wide — 2-Wide Fetch, Decode, and Dispatch

June 15, 2026

Contents

Week 5 in 60 Seconds	4
1 Week 5 — Going Wide	6
2 Comprehensive Theory 1: Going Wide — The Conceptual Shift [~15 min]	8
2.1 What “2-Wide” Means at Each Stage	8
2.2 Why 2 and Not 4?	8
2.3 The Two Slots: A and B	8
2.4 What Stays the Same	9
2.5 What Changes Significantly	9
3 Comprehensive Theory 2: Fetch and Decode [~15 min]	10
3.1 Fetch	10
3.2 Alignment Concerns (and Why We Ignore Them)	10
3.3 Decode	11
3.4 Why Decode Is the Easy Stage	11
4 Comprehensive Theory 3: Intra-Bundle Dependencies [~35 min]	13
4.1 The Problem in One Sentence	13
4.2 Case 1: B Reads What A Writes (RAW Within Bundle)	13
4.3 Case 2: A and B Both Write the Same Register (WAW Within Bundle)	14
4.4 Case 3: B Reads What A Reads (No Dependence, but Both Sources Same)	14
4.5 Case 4: Slot A Is a Branch That Squashes Slot B	15
4.6 Case 5: Slot A or Slot B Cannot Dispatch (Stalls)	15
4.7 Putting All Cases Together	15
4.8 The Worked Example	16
5 Comprehensive Theory 4: 2-Wide Dispatch and the ROB Interface [~15 min]	18
5.1 What Dispatch Does	18
5.2 Stall Coordination	18
5.3 The Stubbed ROB Interface	19
5.4 The Dispatch-to-RS Interface	19
6 Verilog Implementation: Fetch, Decode, 2-Wide Rename, Dispatch [~1 hour reading]	21
6.1 Module: fetch.v	21
6.2 Module: decode.v	22
6.3 Module: rename_unit.v (Widened)	25
6.4 Module: dispatch.v	29
6.5 Notes on the Widened RS	32

7	The Integration Test [~1 hour]	33
7.1	The Test Program	33
7.2	The Testbench Structure	33
7.3	What to Verify Visually in the Waveform	34
7.4	Expected Final State	34
8	Week 5 Deliverables	36
A	Appendix: Solutions to Concept Checks	38
B	Appendix B: Python Helper to Generate test1.hex	41

Week 5 in 60 Seconds

A one-page summary. Theory load drops sharply this week; coding load is heavy. This is where PRAVAH stops being a collection of modules and starts being a pipeline.

The conceptual shift: Every previous module assumed single-issue (one instruction per cycle). This week we widen everything to 2-wide — two instructions flow through fetch, decode, and dispatch in lockstep, every cycle. From an outsider’s view, the pipeline now has “slot A” and “slot B” parallel paths at every stage.

The three new modules:

- **Fetch:** read two instructions per cycle from instruction memory, plus the predicted next PC.
- **Decode:** two independent decoders, each producing a micro-op (extracted opcode, source arch regs, destination arch reg, immediate, instruction type).
- **Dispatch:** the bridge from front-end to back-end. Calls the (now 2-wide) rename unit, allocates ROB entries, sends operations to reservation stations.

The one hard problem: Intra-bundle dependencies. When slot A and slot B dispatch in the same cycle:

- If both write the same arch register: only slot B’s mapping wins in the rename map. But slot A still gets its own physical register.
- If slot B reads what slot A writes: B must *not* use the stale rename map entry. It must bypass — use the physical register A just allocated.
- If slot A is a taken branch: slot B’s instruction must be squashed (it’s on the wrong path).

This bypass logic is the single most common Week 5 bug. The document spends 4 pages on it (Section 3); read carefully.

What we keep from previous weeks:

- Register file: same module, already 4R2W (Week 3). Now actually feeding both dispatch slots.
- Rename unit: **must be widened** from 1-wide to 2-wide. The bypass logic for intra-bundle dependencies lives here.
- Reservation stations: same module (Week 4), but dispatch can now write *two* RS entries per cycle. The RS module port list grows.

What we defer to Week 6:

- ROB Verilog (we use a stub interface this week).

- Branch prediction logic (we hard-code “predict not-taken” for now).
- Wakeup-select for multiple FUs (still single-ALU this week).
- Mispredict recovery / flush.

End-of-week deliverable: Run a 10-instruction test program through fetch → decode → rename → dispatch, and capture a ModelSim waveform showing two instructions being dispatched per cycle in the common case. The execution side is still stubbed; you’ll see RS entries fill but they won’t execute (that needs Week 6’s full integration). What you’re verifying is that the *front-end* works.

Suggested reading order: Section 1 is the conceptual shift to wide pipelines. Sections 2–4 are the three theory pieces (fetch, decode, intra-bundle dependency). Section 5 is the Verilog for the three modules. Section 6 is the integration test. Section 7 is deliverables.

1 Week 5 — Going Wide

Goal

Build the 2-wide front-end of PRAVAH: fetch, decode, and dispatch. By the end of the week, two instructions per cycle flow from instruction memory all the way to reservation station entries, correctly handling the case where the second instruction depends on the first.

Mentee Work (10 hours)

- Read Sections 1–4 (theory, ~2 hours). The intra-bundle section (3) is the conceptual core.
- Read Section 5 (Verilog implementations, ~1 hour).
- Widen the existing `rename_unit.v` from single-issue to 2-wide (~2 hours). This is the trickiest part of the week.
- Implement `fetch.v`, `decode.v`, `dispatch.v` (~3–4 hours).
- Run the 10-instruction integration test (~1 hour).
- Update `learning_log.md`.

Deliverables

- `rtl/fetch.v` — 2-wide instruction fetch.
- `rtl/decode.v` — 2-wide parallel decode.
- `rtl/dispatch.v` — 2-wide dispatch (handles intra-bundle bypass).
- `rtl/rename_unit.v` — widened to 2-wide (replaces Week 3 version).
- `tb/tb_frontend.v` — integration test running a 10-instruction program.
- `sim/waveforms/frontend_dispatch.png` — ModelSim screenshot.
- `programs/test1.hex` — the 10-instruction hex file.
- Updated `learning_log.md`.

What Mentor Will Look For

- Intra-bundle dependency correctly handled (the canonical test: `add x5, x1, x2 / sub x6, x5, x3` dispatched in the same cycle — the `sub` must use the `add`'s freshly allocated physical register, not the stale rename map).
- Same-cycle WAW: `add x5, x1, x2 / add x5, x3, x4` both dispatch; both get distinct physical destinations; rename map ends up pointing to slot B's destination.
- Stall propagation: if the free list has only 1 slot and 2 instructions want to dispatch, slot B stalls and the front-end holds.
- Plain Verilog only.
- Front-end runs the integration test without errors in ModelSim.

2 Comprehensive Theory 1: Going Wide — The Conceptual Shift [~15 min]

2.1 What “2-Wide” Means at Each Stage

Up to this point, every diagram has shown one instruction flowing through one stage at a time. From Week 5 onward, every stage has two parallel lanes:

- **Fetch:** reads *two* 32-bit instructions per cycle from instruction memory. The memory interface must deliver 64 bits per cycle.
- **Decode:** two decoders running independently, each producing a micro-op.
- **Rename:** the rename map must do 6 reads per cycle (2 instructions $\times$ 3 reads each = 2 sources + 1 old destination) and 2 writes per cycle.
- **Dispatch:** two RS entries allocated per cycle, two ROB entries allocated per cycle.

The **back-end** (execute, complete, commit) is not necessarily 2-wide — a 2-wide superscalar with 2 ALUs can still execute two ALU operations per cycle, but if both slots dispatch a multiply, only one MUL is available, so they execute serially. The execute width depends on FU mix, not on the issue width.

2.2 Why 2 and Not 4?

Going wider has diminishing returns:

- **Dependence:** typical code has a dependence every ~3 instructions. A 4-wide machine on typical code finds at most 2 truly independent instructions per cycle on average.
- **Rename complexity:** 4-wide rename needs 12 read ports on the rename map plus a 4 $\times$ 4 cross-check for intra-bundle dependencies. Quadratic complexity.
- **Wakeup-select complexity:** 4-wide select requires picking 4 ready RSs per cycle, which is expensive.

Real high-performance processors do go wider (Intel Skylake is 4–6 wide), but at significant cost. For PRAVAH, 2-wide is the right balance: visibly superscalar, but tractable to debug.

2.3 The Two Slots: A and B

We’ll consistently refer to the two parallel paths as **slot A** and **slot B**, where slot A holds the instruction that is *earlier in program order*. This matters because:

- If both write the same arch register, slot B's write is the architecturally-final one. (The instruction in slot B was issued later by the programmer; in program order, B's write overwrites A's.)
- If slot A is a taken branch, slot B is on the wrong path — it must be squashed.
- If both instructions need to commit, slot A commits first.

In fetch, slot A is the instruction at the current PC and slot B is the one at PC+4. This convention propagates through every stage.

2.4 What Stays the Same

Most of what you've built doesn't change conceptually — only its port count grows:

- The register file (Week 3) is already 4R/2W. Sized correctly.
- Reservation stations (Week 4) are already a multi-entry array. We just need a second dispatch port to write two entries per cycle.
- The PRF ready-bit mechanism is unchanged.
- The rename map data structure (32 entries, one phys reg per arch reg) is unchanged. Only the access pattern changes.

2.5 What Changes Significantly

- **Rename unit logic:** requires intra-bundle bypass (Section 3). This is genuinely new complexity.
- **Dispatch logic:** must coordinate two dispatches in lockstep, including the case where one slot stalls and the other doesn't.
- **Free list:** pops two per cycle (or one, or zero, depending on how many slots write). The pointer arithmetic gets a touch more interesting.

Concept Check 1

1. If PRAVAH's IPC ceiling is 2.0 (2-wide), what's the IPC ceiling of a 4-wide machine? Of an infinitely wide machine? (Hint: think about the IPC of perfectly independent code vs. typical code.)
2. Why is the rename map's read-port count 6 in a 2-wide machine, not 4 or 8?
3. In what real-world scenario would slot A and slot B be *completely independent* (no dependencies, no shared writes, no branches)? Give an example sequence.

3 Comprehensive Theory 2: Fetch and Decode [~15 min]

These two stages are conceptually straightforward. The real challenge is in dispatch and rename (Section 3).

3.1 Fetch

Fetch reads two instructions per cycle from instruction memory. For PRAVAH (which uses a simple combinational instruction memory with a 64-bit read port), this is:

1. Output: `pc_o` (the current PC), drives the address pins of the instruction memory.
2. Read: memory returns 64 bits = two consecutive 32-bit instructions.
3. Slot A = `memory[PC]` = the first 32 bits.
4. Slot B = `memory[PC+4]` = the next 32 bits.
5. Update PC: assuming no branch this cycle, $PC \rightarrow PC + 8$.

Branch handling in fetch: In Week 6 we'll add branch prediction. For Week 5, we keep things simple:

- Predict: not-taken. So PC always advances by 8.
- If a branch is later detected (Week 6 will handle this), we'll redirect PC.

3.2 Alignment Concerns (and Why We Ignore Them)

In real processors, fetching 2 instructions per cycle has an alignment problem: what if the current PC is misaligned with the memory's natural 64-bit boundary? Suppose $PC = 0x4$ (instruction at offset 1 within a 64-bit word). The first cycle can only deliver one instruction; the next 32-bit slice is in the next memory word.

Solutions in real processors:

- Instruction buffer: fetch always a full 64-bit word; buffer the residue.
- Aligned fetch: restrict branch targets to even-aligned (8-byte) boundaries.
- Two-port instruction memory: pay the hardware cost to fetch across boundaries.

PRAVAH's choice: We require the test programs to begin at $PC=0$ and consist of an even number of instructions, with all branch targets at 8-byte boundaries. This is the simplest possible assumption and lets us skip the alignment hardware entirely. It's a real limitation, but for our small test programs it costs nothing.

3.3 Decode

Each decoder takes one 32-bit RV32I instruction word and produces a micro-op containing:

- **op**: opcode (one of our 13 supported instructions).
- **rs1_arch**: architectural source register 1 (5 bits).
- **rs2_arch**: architectural source register 2 (5 bits).
- **rd_arch**: architectural destination register (5 bits).
- **writes_rd**: 1 if this instruction writes a destination; 0 for stores and branches.
- **has_imm**: 1 if the instruction has an immediate (ADDI, branches, loads, stores, JAL).
- **imm**: 32-bit sign-extended immediate.
- **is_branch**: 1 if this is a branch (BEQ, BNE, JAL).
- **is_memory**: 1 if this is LW or SW.
- **fu_type**: which functional unit (ALU, MUL, LSU).
- **valid**: 1 if the decoded instruction is a real instruction (not a NOP padding or squashed slot).

The decoder is purely combinational. Given a 32-bit instruction word, it produces these fields in zero time. Implementation is a big **case** statement on the opcode and funct3/funct7 fields.

Two decoders run in parallel. They are completely independent — no cross-talk between slot A’s decoder and slot B’s decoder. (The interaction happens in dispatch, not decode.)

3.4 Why Decode Is the Easy Stage

Decode is the cheapest stage in the pipeline. It’s purely combinational, has no state, and the logic is trivial: extract bit fields based on opcode. Real processors decode 4+ instructions per cycle without breaking a sweat. The complexity is elsewhere.

Concept Check 2

1. In RV32I, the instruction `addi x5, x6, 100` is encoded with the immediate field in bits [31:20]. Why must the decoder sign-extend the immediate?
2. Suppose the decoder encounters a 32-bit value that doesn't correspond to any supported instruction. What should the decoder do? Why?
3. If decode is purely combinational, what happens in the same clock cycle: fetch reads memory, decode processes the result, and then? Where does the boundary between decode and dispatch sit in time?

4 Comprehensive Theory 3: Intra-Bundle Dependencies [~35 min]

This is the conceptual nucleus of Week 5. If you understand this section, the Verilog will be straightforward. If you don't, your code will silently produce wrong results.

4.1 The Problem in One Sentence

When two instructions dispatch in the same cycle, slot B may depend on slot A — but the rename map doesn't yet know about slot A's destination, because slot A's update hasn't been clocked in.

Make sure you absorb that sentence. Then we'll walk through three specific cases.

4.2 Case 1: B Reads What A Writes (RAW Within Bundle)

```
slot A: add x5, x1, x2      # writes x5
slot B: sub x6, x5, x3      # reads x5 (depends on A!)
```

What the naive rename unit does:

- Slot A: read `rename_map[5]` (= P5, the old mapping) into A's `old_phys_dest`. Pop free list, get P32. Update `rename_map[5] → P32` (at end of cycle).
- Slot B: read `rename_map[5]` (= P5 *still*, because A's update isn't yet visible). Use P5 as B's source 1.

This is wrong. B's source 1 was supposed to be “whatever A produces.” That's P32, not P5. If we use P5, B will read the stale value of x5 from before A executed.

The fix: intra-bundle bypass.

- For each of B's source registers, check: does slot A write this register (and is A's `writes_rd` asserted)?
- If yes: use A's newly-allocated physical destination (P32 in our example) as B's source.
- If no: use the rename map lookup as normal.

In Verilog, this is a multiplexer per source:

```
1 // For slot B's source 1
2 wire B_src1_uses_A = slotA_writes_rd & (slotA_arch_dest ==
   slotB_arch_src1);
3 wire [PHYS_W-1:0] B_phys_src1 = B_src1_uses_A ? slotA_phys_dest
4                                           : rename_map[
                                           slotB_arch_src1
                                           ];
```

4.3 Case 2: A and B Both Write the Same Register (WAW Within Bundle)

```
slot A: add x5, x1, x2      # writes x5
slot B: add x5, x6, x7      # ALSO writes x5
```

What's correct? In program order, B is later, so B's write to x5 is the architecturally-visible one. After both instructions commit, x5 should hold B's result.

The mechanism:

- Both A and B get fresh physical destinations from the free list: P32 for A, P33 for B.
- `rename_map[5]` is updated to P33 at end of cycle (B's destination, not A's).
- A's `old_phys_dest` = P5 (the original mapping). A's ROB entry will free P5 when A commits.
- B's `old_phys_dest` = P32 (A's destination!). This is the subtle part. When B commits later, P32 will be freed.

The chain of frees: the ROB will eventually free P5 (when A commits) and then P32 (when B commits). The architectural state evolves correctly: x5 was P5, then P32 (briefly), then P33 (the final value).

Verilog:

```
1 wire AB_write_same = slotA_writes_rd & slotB_writes_rd &
2                       (slotA_arch_dest == slotB_arch_dest);
3
4 // B's old_phys_dest needs the bypass
5 wire [PHYS_W-1:0] B_old_phys_dest = AB_write_same ?
6                               slotA_phys_dest
6                               : rename_map[
6                               slotB_arch_dest
6                               ];
```

4.4 Case 3: B Reads What A Reads (No Dependence, but Both Sources Same)

```
slot A: add x5, x1, x2      # reads x1
slot B: sub x6, x1, x3      # also reads x1 (no dependence)
```

This is the easy case. Both A and B independently read `rename_map[1]`, which returns the same physical register (say P1). Both A and B end up with P1 as their respective source 1. No bypass needed.

The rename map has multiple read ports specifically to support this — two reads of `rename_map[1]` in the same cycle return P1 to both readers without contention.

4.5 Case 4: Slot A Is a Branch That Squashes Slot B

```
slot A: bne x1, x2, target      # branch (may redirect PC)
slot B: add x5, x6, x7          # only executes if A is not taken
```

For Week 5: We don't have branch resolution yet, so we hard-code "not-taken" — B always proceeds. (Week 6 adds the prediction; Week 6 also adds the recovery logic to squash B if A turns out to be taken.)

For Week 5 sanity: If your decoder sees a branch in slot A, you can choose:

- **Conservative:** dispatch only slot A this cycle; hold slot B for the next cycle. Simpler, slightly slower.
- **Aggressive (chosen for PRAVAH):** dispatch both; the prediction is "not-taken," so B is on the predicted path. Mispredict recovery happens later.

We pick aggressive because it's the path forward to a working pipeline, even though it requires Week 6 to be fully correct.

4.6 Case 5: Slot A or Slot B Cannot Dispatch (Stalls)

What if the free list has only 1 slot remaining and both A and B want destinations?

The policy: dispatch slot A. Hold slot B. On the next cycle, retry the (B, next-instruction-after-B) pair.

This means dispatch must coordinate with fetch: if only A dispatched this cycle, fetch must *not* advance to the next pair. It must hold the current pair, with B as the leading instruction next cycle.

Implementation hint: use a "leader pointer" to remember which of the two fetched instructions is the next to dispatch. On the cycle B dispatches but the next pair isn't fetched yet, the front-end is half-stalled. This is fiddly to get right; the testbench will catch bugs.

For PRAVAH's simple case: we'll require the testbench to provide enough free-list slack that we never half-dispatch. Specifically, with a 16-entry free list and only 10 test instructions, we never run out. But your dispatch logic should still handle the half-dispatch case correctly.

4.7 Putting All Cases Together

At dispatch time for a (slot A, slot B) bundle:

1. Decode A and B; determine `writes_rd`, `src` registers, etc.
2. Compute intra-bundle interactions:
 - `B_src1_uses_A` = (A writes rd) & (A.rd_arch == B.src1_arch)
 - `B_src2_uses_A` = (A writes rd) & (A.rd_arch == B.src2_arch)

- `AB_write_same` = (A writes rd) & (B writes rd) & (A.rd_arch == B.rd_arch)
3. Look up rename map for A's sources, A's destination (for `old_phys_dest`), B's sources, B's destination.
 4. Pop free list: A gets first entry, B gets second entry (if both write).
 5. Apply bypass muxes for B's sources and B's `old_phys_dest`.
 6. Update rename map: A's destination $\rightarrow$ A's new phys; B's destination $\rightarrow$ B's new phys. (If `AB_write_same`, only B's update wins; we can either write twice with B winning by ordering or just suppress A's write.)
 7. Stall handling: if free list cannot satisfy both, dispatch A only, hold B.

4.8 The Worked Example

Let's trace through a 4-instruction sequence:

```
I1: add x5, x1, x2      # in slot A of bundle 1
I2: sub x6, x5, x3      # in slot B of bundle 1 (depends on I1!)
I3: and x7, x6, x4      # in slot A of bundle 2 (depends on I2!)
I4: or  x5, x7, x1      # in slot B of bundle 2 (depends on I3; WAW with I1)
```

Initial state: `rename_map` identity ($x_i \rightarrow P_i$), free list = [P32, P33, P34, P35, ...].

Cycle 1: dispatch I1 (slot A), I2 (slot B).

Slot A (I1):

- Sources: $x1 \rightarrow P1$ (read rename map), $x2 \rightarrow P2$.
- Dest: $x5$. Pop free list: P32. `old_phys_dest` = P5 (current map).
- New mapping: $x5 \rightarrow P32$.

Slot B (I2):

- Bypass check: $I2.src1(x5) == I1.dest(x5)$? Yes! `B_src1_uses_A` = 1.
- $I2.src1 \rightarrow P32$ (A's destination). *Not* P5 from the rename map.
- $I2.src2(x3) \rightarrow P3$ (rename map).
- Dest: $x6$. Pop free list: P33. `AB_write_same`? No (different destinations). So B's `old_phys_dest` = `rename_map[6]` = P6.
- New mapping: $x6 \rightarrow P33$.

After cycle 1: `rename_map[5]` = P32, `rename_map[6]` = P33. Free list head advanced by 2.

Cycle 2: dispatch I3 (slot A), I4 (slot B).

Slot A (I3):

- Sources: $x6 \rightarrow P33$ (read rename map, reflects last cycle's I2 update), $x4 \rightarrow P4$.
- Dest: $x7$. Pop free list: P34. `old_phys_dest = P7`.
- New mapping: $x7 \rightarrow P34$.

Slot B (I4):

- Bypass check: `I4.src1 (x7) == I3.dest (x7)?` Yes! `B_src1_uses_A = 1`.
- `I4.src1` $\rightarrow$ P34 (A's destination this cycle).
- `I4.src2 (x1)` $\rightarrow$ P1 (rename map).
- Dest: $x5$. Pop free list: P35. `AB_write_same?` No (I3 writes $x7$, I4 writes $x5$). So B's `old_phys_dest = rename_map[5] = P32` (the result of cycle 1's I1!).
- New mapping: $x5 \rightarrow P35$.

After cycle 2: `rename_map[5] = P35` (I4 won), `rename_map[7] = P34`. The WAW chain (I1 wrote $x5$, then I4 also writes $x5$) is handled correctly: P32 (I1's dest) will be freed when I4 commits, because I4's `old_phys_dest` captured P32 at dispatch.

The bypass machinery just worked. I2 correctly got P32 as its source; I4 correctly got P34. The rename map ended up pointing to I4's destination (P35) for $x5$, exactly as program order requires.

Concept Check 3

1. In Cycle 2 of the worked example, I4's `old_phys_dest` is P32 (I1's destination from Cycle 1). Why is this correct? Walk through what happens when I4 eventually commits.
2. Suppose the bundle (slot A: `add x5, x1, x2`, slot B: `add x5, x3, x4`) dispatches (Case 2). What does `AB_write_same` equal? What does B's `old_phys_dest` equal? Why doesn't A's update to `rename_map` "leak through"?
3. In an aggressive "always not-taken" design, what's the cost of being wrong on a branch in slot A? (Hint: B's effects need to be undone.)

Going deeper: Shen & Lipasti Chapter 5 covers 2-wide rename with explicit treatment of intra-bundle dependencies. BOOM's rename stage source code is also a good reference (in Chisel, but readable).

5 Comprehensive Theory 4: 2-Wide Dispatch and the ROB Interface [~15 min]

Dispatch is the bridge from front-end (fetch, decode) to back-end (RS, ROB). It takes two micro-ops from the decoders, renames them, and writes them to the reservation stations and ROB.

5.1 What Dispatch Does

For each of the two slots (A, B):

1. Call the rename unit with the decoded sources and destination. Receive back: `phys_src1`, `phys_src2`, `phys_dest`, `old_phys_dest`, `stall`.
2. Determine which functional unit this instruction targets (ALU, MUL, LSU). Find a free RS attached to that FU.
3. Write a new RS entry: `{op, phys_src1, phys_src2, phys_dest, rob_idx}`.
4. Allocate a new ROB entry at the tail: `{arch_dest, phys_dest, old_phys_dest, valid, ready=0}`.
5. Advance the ROB tail by 2 (if both slots dispatched) or 1 (if only A) or 0 (if neither).

5.2 Stall Coordination

Dispatch can be stalled by:

- **Free list empty:** the rename unit signals stall.
- **ROB full:** no free ROB entry to allocate.
- **RS array full:** no free RS slot for the target FU.

Policy:

- If only A is dispatchable: dispatch A, hold B.
- If neither is dispatchable: stall both, hold the bundle, signal the front-end to not advance fetch.

When dispatch stalls, the front-end must also stall (fetch must not advance to the next pair, decode must hold its current output). This is communicated by a `fe_stall_o` signal flowing backward from dispatch to fetch.

5.3 The Stubbed ROB Interface

We don't build the ROB this week (that's Week 6). But dispatch needs to write to it, so we define an interface as a stub:

```
1 // ROB stub interface (driven by dispatch, consumed by Week 6's
  ROB)
2 output reg                rob_alloc_A_o;
3 output reg [PHYS_ADDR_W-1:0] rob_A_phys_dest_o;
4 output reg [PHYS_ADDR_W-1:0] rob_A_old_phys_dest_o;
5 output reg [ARCH_ADDR_W-1:0] rob_A_arch_dest_o;
6 output reg                rob_A_writes_rd_o;
7
8 output reg                rob_alloc_B_o;
9 output reg [PHYS_ADDR_W-1:0] rob_B_phys_dest_o;
10 output reg [PHYS_ADDR_W-1:0] rob_B_old_phys_dest_o;
11 output reg [ARCH_ADDR_W-1:0] rob_B_arch_dest_o;
12 output reg                rob_B_writes_rd_o;
13
14 input  wire [ROB_ADDR_W-1:0] rob_tail_i;      // current ROB
    tail
15 input  wire                rob_full_i;        // 1 if no free
    ROB entries
```

In Week 6, the ROB module will provide `rob_tail_i` and `rob_full_i` and consume the alloc signals. For Week 5, the testbench provides them: `rob_tail_i` is a counter that increments by the number of allocations per cycle, and `rob_full_i` is 0 (assume infinite ROB for testing).

5.4 The Dispatch-to-RS Interface

Each slot of dispatch writes to one RS. The RS array (Week 4) was built with one dispatch port; for Week 5 we need to extend it to two dispatch ports.

Two options:

- **Modify the RS module** to have two dispatch ports. Cleaner long-term.
- **Instantiate two RS arrays** (one for slot A, one for slot B). Simpler for Week 5 but wastes slots.

For PRAVAH we pick option 1. Section 5 shows the widened RS port list.

Concept Check 4

1. In a cycle where dispatch can only place A and not B (free list has one slot left), what must the front-end do next cycle to ensure B isn't lost?
2. Why does dispatch need both `phys_dest` *and* `old_phys_dest` to send to the ROB? (Hint: think about commit and flush.)
3. Suppose the RS array has only 2 free slots and both must come from the ALU RS pool. If both A and B are ALU operations, can dispatch proceed?

6 Verilog Implementation: Fetch, Decode, 2-Wide Rename, Dispatch [~1 hour reading]

We have four modules to build (or widen) this week. We'll cover the design of each, then the integration testbench in Section 6.

6.1 Module: fetch.v

```
1 module fetch #(
2     parameter PC_WIDTH      = 32,
3     parameter IMEM_DEPTH    = 256          // 256 instructions = 1 KB
4 ) (
5     input  wire              clk_i,
6     input  wire              reset_i,
7     input  wire              fe_stall_i,    // from dispatch
8
9     // Outputs to decode
10    output reg [PC_WIDTH-1:0] pc_A_o,
11    output reg [31:0]         instr_A_o,
12    output reg                valid_A_o,
13    output reg [PC_WIDTH-1:0] pc_B_o,
14    output reg [31:0]         instr_B_o,
15    output reg                valid_B_o
16 );
17
18 // PC register
19 reg [PC_WIDTH-1:0] pc;
20
21 // Instruction memory (combinational read)
22 // Initialized from a hex file in the testbench via $readmemh
23 reg [31:0] imem [0:IMEM_DEPTH-1];
24
25 // Combinational instruction reads
26 wire [PC_WIDTH-1:0] pc_next_A = pc;
27 wire [PC_WIDTH-1:0] pc_next_B = pc + 4;
28 wire [31:0] instr_A_comb = imem[pc[PC_WIDTH-1:2]];
29 wire [31:0] instr_B_comb = imem[(pc + 4) >> 2];
30
31 // Validity: both instructions valid unless we've fetched
32 // past end
33 wire valid_A_comb = (pc_next_A[PC_WIDTH-1:2] < IMEM_DEPTH);
34 wire valid_B_comb = (pc_next_B[PC_WIDTH-1:2] < IMEM_DEPTH);
35
36 // Sequential update
37 always @(posedge clk_i) begin
```

```

37     if (reset_i) begin
38         pc <= 0;
39         pc_A_o <= 0; instr_A_o <= 0; valid_A_o <= 0;
40         pc_B_o <= 0; instr_B_o <= 0; valid_B_o <= 0;
41     end else if (~fe_stall_i) begin
42         // Advance: latch this cycle's fetch, advance PC by 8
43         pc <= pc + 8;
44         pc_A_o <= pc_next_A;
45         instr_A_o <= instr_A_comb;
46         valid_A_o <= valid_A_comb;
47         pc_B_o <= pc_next_B;
48         instr_B_o <= instr_B_comb;
49         valid_B_o <= valid_B_comb;
50     end
51     // If stalled, hold output (already latched)
52 end
53
54 endmodule

```

Notes:

- The imem is read combinationaly; the latched outputs go to decode next cycle.
- Stall handling: if `fe_stall_i` is 1, PC doesn't advance and outputs stay frozen. This is the back-pressure mechanism.
- No branch prediction yet; PC just advances by 8 each cycle.
- Validity bit: if we've fetched past the end of the instruction memory, the slot is marked invalid. Decode will then mark its output as invalid (no real instruction).

6.2 Module: decode.v

```

1 module decode #(
2     parameter ARCH_ADDR_W = 5,
3     parameter OP_WIDTH    = 4
4 ) (
5     // Inputs from fetch
6     input wire [31:0]      instr_i,
7     input wire             valid_i,
8
9     // Outputs to dispatch (one decoded micro-op)
10    output wire [OP_WIDTH-1:0] op_o,
11    output wire [ARCH_ADDR_W-1:0] rs1_arch_o,
12    output wire [ARCH_ADDR_W-1:0] rs2_arch_o,
13    output wire [ARCH_ADDR_W-1:0] rd_arch_o,

```

```

14     output wire                writes_rd_o,
15     output wire [31:0]         imm_o,
16     output wire                is_branch_o,
17     output wire                is_memory_o,
18     output wire [1:0]          fu_type_o,    // 0=ALU, 1=MUL
19     , 2=LSU
20     output wire                valid_o
21 );
22
23 // RV32I encoding fields
24 wire [6:0] opcode = instr_i[6:0];
25 wire [4:0] rd      = instr_i[11:7];
26 wire [2:0] funct3  = instr_i[14:12];
27 wire [4:0] rs1     = instr_i[19:15];
28 wire [4:0] rs2     = instr_i[24:20];
29 wire [6:0] funct7  = instr_i[31:25];
30
31 // Decoded op (our 4-bit internal encoding)
32 // 0=ADD 1=SUB 2=AND 3=OR 4=XOR 5=SLL 6=SRL 7=SLT
33 // 8=ADDI 9=LW 10=SW 11=BEQ 12=BNE 13=JAL
34 reg [OP_WIDTH-1:0] op_internal;
35
36 always @* begin
37     case (opcode)
38         7'b0110011: begin // R-type
39             case (funct3)
40                 3'b000: op_internal = (funct7[5]) ? 4'd1 : 4'
41                     d0; // SUB or ADD
42                 3'b111: op_internal = 4'd2; // AND
43                 3'b110: op_internal = 4'd3; // OR
44                 3'b100: op_internal = 4'd4; // XOR
45                 3'b001: op_internal = 4'd5; // SLL
46                 3'b101: op_internal = 4'd6; // SRL
47                 3'b010: op_internal = 4'd7; // SLT
48                 default: op_internal = 4'd0;
49             endcase
50         end
51         7'b0010011: op_internal = 4'd8; // ADDI (I-type)
52         7'b0000011: op_internal = 4'd9; // LW
53         7'b0100011: op_internal = 4'd10; // SW
54         7'b1100011: op_internal = (funct3[0]) ? 4'd12 : 4'd11
55             ; // BNE or BEQ
56         7'b1101111: op_internal = 4'd13; // JAL
57         default:    op_internal = 4'd0;
58     endcase

```

```

56     end
57
58     // Output assignments
59     assign op_o          = op_internal;
60     assign rs1_arch_o    = rs1;
61     assign rs2_arch_o    = rs2;
62     assign rd_arch_o     = rd;
63
64     // Does this instruction write rd?
65     assign writes_rd_o = valid_i & (
66         (opcode == 7'b0110011) | // R-type
67         (opcode == 7'b0010011) | // ADDI
68         (opcode == 7'b0000011) | // LW
69         (opcode == 7'b1101111)   // JAL
70     );
71
72     assign is_branch_o = valid_i & (
73         (opcode == 7'b1100011) | // BEQ/BNE
74         (opcode == 7'b1101111)   // JAL
75     );
76
77     assign is_memory_o = valid_i & (
78         (opcode == 7'b0000011) | // LW
79         (opcode == 7'b0100011)   // SW
80     );
81
82     assign fu_type_o = is_memory_o ? 2'd2 : 2'd0; // ALU or LSU
83     (no MUL in our subset)
84
85     // Sign-extended immediate
86     wire [31:0] imm_I = {{20{instr_i[31]}}, instr_i[31:20]};
87     wire [31:0] imm_S = {{20{instr_i[31]}}, instr_i[31:25],
88         instr_i[11:7]};
89     wire [31:0] imm_B = {{19{instr_i[31]}}, instr_i[31], instr_i
90         [7],
91         instr_i[30:25], instr_i[11:8], 1'b0};
92     wire [31:0] imm_J = {{11{instr_i[31]}}, instr_i[31], instr_i
93         [19:12],
94         instr_i[20], instr_i[30:21], 1'b0};
95
96     assign imm_o = (opcode == 7'b0100011) ? imm_S :
97         (opcode == 7'b1100011) ? imm_B :
98         (opcode == 7'b1101111) ? imm_J :
99         imm_I; // default to I-type

```

```

97     assign valid_o = valid_i;
98
99 endmodule

```

Note: Two copies of this decoder are instantiated, one for slot A and one for slot B. They are completely independent.

6.3 Module: rename_unit.v (Widened)

The Week 3 rename unit was single-issue. We must widen it to 2-wide with all the intra-bundle bypass logic.

```

1 module rename_unit #(
2     parameter NUM_ARCH_REGS = 32,
3     parameter NUM_PHYS_REGS = 48,
4     parameter ARCH_ADDR_W   = 5,
5     parameter PHYS_ADDR_W   = 6,
6     parameter FREE_LIST_SZ  = NUM_PHYS_REGS - NUM_ARCH_REGS
7 ) (
8     input  wire          clk_i,
9     input  wire          reset_i,
10
11     // Slot A inputs
12     input  wire          disp_A_valid_i,
13     input  wire [ARCH_ADDR_W-1:0] disp_A_rs1_arch_i,
14     input  wire [ARCH_ADDR_W-1:0] disp_A_rs2_arch_i,
15     input  wire [ARCH_ADDR_W-1:0] disp_A_rd_arch_i,
16     input  wire          disp_A_writes_rd_i,
17     output wire [PHYS_ADDR_W-1:0] disp_A_rs1_phys_o,
18     output wire [PHYS_ADDR_W-1:0] disp_A_rs2_phys_o,
19     output wire [PHYS_ADDR_W-1:0] disp_A_rd_phys_o,
20     output wire [PHYS_ADDR_W-1:0] disp_A_rd_old_phys_o,
21
22     // Slot B inputs
23     input  wire          disp_B_valid_i,
24     input  wire [ARCH_ADDR_W-1:0] disp_B_rs1_arch_i,
25     input  wire [ARCH_ADDR_W-1:0] disp_B_rs2_arch_i,
26     input  wire [ARCH_ADDR_W-1:0] disp_B_rd_arch_i,
27     input  wire          disp_B_writes_rd_i,
28     output wire [PHYS_ADDR_W-1:0] disp_B_rs1_phys_o,
29     output wire [PHYS_ADDR_W-1:0] disp_B_rs2_phys_o,
30     output wire [PHYS_ADDR_W-1:0] disp_B_rd_phys_o,
31     output wire [PHYS_ADDR_W-1:0] disp_B_rd_old_phys_o,
32
33     // Stall (1 if can't allocate enough phys regs this cycle)

```

```

34     output wire                                stall_A_o,      // can't
        allocate even one
35     output wire                                stall_B_o,      // can't
        allocate two
36
37     // Commit port (return phys reg to free list)
38     input  wire                                commit_valid_i,
39     input  wire [PHYS_ADDR_W-1:0]             commit_old_phys_i
40 );
41
42     // Rename map
43     reg [PHYS_ADDR_W-1:0] rename_map [0:NUM_ARCH_REGS-1];
44
45     // Free list
46     reg [PHYS_ADDR_W-1:0] free_list [0:FREE_LIST_SZ-1];
47     reg [$clog2(FREE_LIST_SZ):0] fl_head, fl_tail;
48     reg [$clog2(FREE_LIST_SZ):0] fl_count;
49
50     // ----- A and B need-to-allocate signals -----
51     wire A_needs_alloc = disp_A_valid_i & disp_A_writes_rd_i;
52     wire B_needs_alloc = disp_B_valid_i & disp_B_writes_rd_i;
53
54     wire allocs_needed_2 = A_needs_alloc & B_needs_alloc;
55     wire allocs_needed_1 = A_needs_alloc ^ B_needs_alloc;
56
57     // Stall: not enough free regs
58     assign stall_A_o = disp_A_valid_i & A_needs_alloc & (fl_count
        == 0);
59     assign stall_B_o = disp_B_valid_i & allocs_needed_2 & (
        fl_count < 2);
60
61     // ----- Compute physical destinations -----
62     // A always gets fl_head (if it allocates)
63     wire [PHYS_ADDR_W-1:0] A_new_phys = free_list[fl_head[$clog2(
        FREE_LIST_SZ)-1:0]];
64
65     // B gets fl_head + 1 (if both allocate) or fl_head (if only
        B allocates)
66     wire [PHYS_ADDR_W-1:0] B_new_phys = A_needs_alloc ?
        free_list[(fl_head + 1) % FREE_LIST_SZ] :
67     free_list[fl_head[$clog2(FREE_LIST_SZ)-1:0]];
68
69
70     // ----- Intra-bundle bypass for slot B's sources -----
71     wire B_src1_uses_A = A_needs_alloc & (disp_A_rd_arch_i ==
        disp_B_rs1_arch_i);

```

```

72  wire B_src2_uses_A = A_needs_alloc & (disp_A_rd_arch_i ==
    disp_B_rs2_arch_i);
73  wire AB_write_same = A_needs_alloc & B_needs_alloc & (
    disp_A_rd_arch_i == disp_B_rd_arch_i);
74
75  // ----- Slot A's outputs -----
76  assign disp_A_rs1_phys_o    = rename_map[disp_A_rs1_arch_i];
77  assign disp_A_rs2_phys_o    = rename_map[disp_A_rs2_arch_i];
78  assign disp_A_rd_phys_o     = A_needs_alloc ? A_new_phys :
79                                rename_map[
                                    disp_A_rd_arch_i
                                ];
80  assign disp_A_rd_old_phys_o = rename_map[disp_A_rd_arch_i];
81
82  // ----- Slot B's outputs (with bypass) -----
83  // B's source 1: if it depends on A, use A_new_phys; else use
    rename map
84  assign disp_B_rs1_phys_o = B_src1_uses_A ? A_new_phys
85                                : rename_map[
                                    disp_B_rs1_arch_i
                                ];
86  assign disp_B_rs2_phys_o = B_src2_uses_A ? A_new_phys
87                                : rename_map[
                                    disp_B_rs2_arch_i
                                ];
88  assign disp_B_rd_phys_o    = B_needs_alloc ? B_new_phys :
89                                rename_map[
                                    disp_B_rd_arch_i
                                ];
90  assign disp_B_rd_old_phys_o = AB_write_same ? A_new_phys
91                                : rename_map[
                                    disp_B_rd_arch_i
                                ];
92
93  // ----- Helper: actual allocations this cycle -----
94  wire do_alloc_A = A_needs_alloc & ~stall_A_o;
95  wire do_alloc_B = B_needs_alloc & ~stall_B_o & ~stall_A_o;
96  wire do_commit  = commit_valid_i;
97
98  // ----- Synchronous updates -----
99  integer i;
100 always @(posedge clk_i) begin
101     if (reset_i) begin
102         for (i = 0; i < NUM_ARCH_REGS; i = i + 1)
103             rename_map[i] <= i[PHYS_ADDR_W-1:0];

```

```

104     for (i = 0; i < FREE_LIST_SZ; i = i + 1)
105         free_list[i] <= NUM_ARCH_REGS[PHYS_ADDR_W-1:0] +
            i[PHYS_ADDR_W-1:0];
106     fl_head <= 0;
107     fl_tail <= 0;
108     fl_count <= FREE_LIST_SZ[$clog2(FREE_LIST_SZ):0];
109 end else begin
110     // Update rename map
111     // A's update (if it allocates)
112     if (do_alloc_A) begin
113         rename_map[disp_A_rd_arch_i] <= A_new_phys;
114     end
115     // B's update (if it allocates) -- B wins if
        AB_write_same
116     if (do_alloc_B) begin
117         rename_map[disp_B_rd_arch_i] <= B_new_phys;
118     end
119
120     // Advance free list head
121     if (do_alloc_A & do_alloc_B)
122         fl_head <= (fl_head + 2) % FREE_LIST_SZ;
123     else if (do_alloc_A | do_alloc_B)
124         fl_head <= (fl_head + 1) % FREE_LIST_SZ;
125
126     // Commit: push to free list
127     if (do_commit) begin
128         free_list[fl_tail[$clog2(FREE_LIST_SZ)-1:0]] <=
            commit_old_phys_i;
129         fl_tail <= (fl_tail + 1) % FREE_LIST_SZ;
130     end
131
132     // Free list count update (single clean expression)
133     fl_count <= fl_count
134         - (do_alloc_A ? 1 : 0)
135         - (do_alloc_B ? 1 : 0)
136         + (do_commit ? 1 : 0);
137 end
138 end
139
140 endmodule

```

Key bypass logic to verify by hand:

- B_src1_uses_A: only true if A allocates a destination AND that destination matches B's source 1 architectural register.

- B_src2_uses_A: same for source 2.
- AB_write_same: only matters for B's old_phys_dest. If both write the same arch reg, B's old_phys_dest is A's freshly-allocated destination.

6.4 Module: dispatch.v

Dispatch is mostly wiring — it takes outputs from decode and rename and routes them to RS and ROB.

```

1 module dispatch #(
2     parameter ARCH_ADDR_W = 5,
3     parameter PHYS_ADDR_W = 6,
4     parameter OP_WIDTH     = 4,
5     parameter ROB_ADDR_W   = 3
6 ) (
7     input wire          clk_i,
8     input wire          reset_i,
9
10    // From decode (slot A)
11    input wire           decode_A_valid_i,
12    input wire [OP_WIDTH-1:0] decode_A_op_i,
13    input wire [ARCH_ADDR_W-1:0] decode_A_rs1_arch_i,
14    input wire [ARCH_ADDR_W-1:0] decode_A_rs2_arch_i,
15    input wire [ARCH_ADDR_W-1:0] decode_A_rd_arch_i,
16    input wire           decode_A_writes_rd_i,
17    input wire [1:0]      decode_A_fu_type_i,
18
19    // From decode (slot B), similar...
20    input wire           decode_B_valid_i,
21    input wire [OP_WIDTH-1:0] decode_B_op_i,
22    input wire [ARCH_ADDR_W-1:0] decode_B_rs1_arch_i,
23    input wire [ARCH_ADDR_W-1:0] decode_B_rs2_arch_i,
24    input wire [ARCH_ADDR_W-1:0] decode_B_rd_arch_i,
25    input wire           decode_B_writes_rd_i,
26    input wire [1:0]      decode_B_fu_type_i,
27
28    // From rename (slot A's physical regs)
29    input wire [PHYS_ADDR_W-1:0] rename_A_rs1_phys_i,
30    input wire [PHYS_ADDR_W-1:0] rename_A_rs2_phys_i,
31    input wire [PHYS_ADDR_W-1:0] rename_A_rd_phys_i,
32    input wire [PHYS_ADDR_W-1:0] rename_A_rd_old_phys_i,
33    input wire           rename_stall_A_i,
34
35    // From rename (slot B)
36    input wire [PHYS_ADDR_W-1:0] rename_B_rs1_phys_i,

```

```

37     input  wire [PHYS_ADDR_W-1:0]      rename_B_rs2_phys_i ,
38     input  wire [PHYS_ADDR_W-1:0]      rename_B_rd_phys_i ,
39     input  wire [PHYS_ADDR_W-1:0]      rename_B_rd_old_phys_i ,
40     input  wire                          rename_stall_B_i ,
41
42     // ROB interface
43     input  wire [ROB_ADDR_W-1:0]        rob_tail_i ,
44     input  wire                          rob_full_i ,
45     output wire                          rob_alloc_A_o ,
46     output wire [ARCH_ADDR_W-1:0]        rob_A_arch_dest_o ,
47     output wire [PHYS_ADDR_W-1:0]        rob_A_phys_dest_o ,
48     output wire [PHYS_ADDR_W-1:0]        rob_A_old_phys_dest_o ,
49     output wire                          rob_A_writes_rd_o ,
50     output wire                          rob_alloc_B_o ,
51     output wire [ARCH_ADDR_W-1:0]        rob_B_arch_dest_o ,
52     output wire [PHYS_ADDR_W-1:0]        rob_B_phys_dest_o ,
53     output wire [PHYS_ADDR_W-1:0]        rob_B_old_phys_dest_o ,
54     output wire                          rob_B_writes_rd_o ,
55
56     // To RS (slot A)
57     output wire                          rs_disp_A_valid_o ,
58     output wire [OP_WIDTH-1:0]           rs_disp_A_op_o ,
59     output wire [PHYS_ADDR_W-1:0]        rs_disp_A_pj_o ,
60     output wire [PHYS_ADDR_W-1:0]        rs_disp_A_pk_o ,
61     output wire [PHYS_ADDR_W-1:0]        rs_disp_A_pd_o ,
62     output wire [ROB_ADDR_W-1:0]         rs_disp_A_rob_idx_o ,
63
64     // To RS (slot B)
65     output wire                          rs_disp_B_valid_o ,
66     output wire [OP_WIDTH-1:0]           rs_disp_B_op_o ,
67     output wire [PHYS_ADDR_W-1:0]        rs_disp_B_pj_o ,
68     output wire [PHYS_ADDR_W-1:0]        rs_disp_B_pk_o ,
69     output wire [PHYS_ADDR_W-1:0]        rs_disp_B_pd_o ,
70     output wire [ROB_ADDR_W-1:0]         rs_disp_B_rob_idx_o ,
71
72     // Back-pressure to fetch
73     output wire                          fe_stall_o
74 );
75
76     // Combine all stall sources
77     wire any_stall = rename_stall_A_i | rename_stall_B_i |
78                     rob_full_i;
79
80     // Slot A dispatches if valid, not stalled
81     wire A_dispatches = decode_A_valid_i & ~rename_stall_A_i & ~

```

```

rob_full_i;

81
82 // Slot B dispatches if valid, not stalled, and A is
   dispatching (else B holds)
83 wire B_dispatches = decode_B_valid_i & ~rename_stall_B_i & ~
   rob_full_i & A_dispatches;
84
85 // Back-pressure: if A can't dispatch, hold fetch
86 // (B-only dispatch is acceptable but rare; we conservatively
   stall fetch if either fails)
87 assign fe_stall_o = (decode_A_valid_i & ~A_dispatches) |
88                     (decode_B_valid_i & ~B_dispatches);
89
90 // ROB outputs
91 assign rob_alloc_A_o      = A_dispatches;
92 assign rob_A_arch_dest_o  = decode_A_rd_arch_i;
93 assign rob_A_phys_dest_o  = rename_A_rd_phys_i;
94 assign rob_A_old_phys_dest_o = rename_A_rd_old_phys_i;
95 assign rob_A_writes_rd_o  = decode_A_writes_rd_i;
96
97 assign rob_alloc_B_o      = B_dispatches;
98 assign rob_B_arch_dest_o  = decode_B_rd_arch_i;
99 assign rob_B_phys_dest_o  = rename_B_rd_phys_i;
100 assign rob_B_old_phys_dest_o = rename_B_rd_old_phys_i;
101 assign rob_B_writes_rd_o  = decode_B_writes_rd_i;
102
103 // RS dispatch outputs
104 assign rs_disp_A_valid_o  = A_dispatches & (
   decode_A_fu_type_i == 2'd0); // ALU only this week
105 assign rs_disp_A_op_o     = decode_A_op_i;
106 assign rs_disp_A_pj_o     = rename_A_rs1_phys_i;
107 assign rs_disp_A_pk_o     = rename_A_rs2_phys_i;
108 assign rs_disp_A_pd_o     = rename_A_rd_phys_i;
109 assign rs_disp_A_rob_idx_o = rob_tail_i;
110
111 assign rs_disp_B_valid_o  = B_dispatches & (
   decode_B_fu_type_i == 2'd0);
112 assign rs_disp_B_op_o     = decode_B_op_i;
113 assign rs_disp_B_pj_o     = rename_B_rs1_phys_i;
114 assign rs_disp_B_pk_o     = rename_B_rs2_phys_i;
115 assign rs_disp_B_pd_o     = rename_B_rd_phys_i;
116 assign rs_disp_B_rob_idx_o = rob_tail_i + (A_dispatches ? 1 :
   0);
117
118 endmodule

```

Dispatch is mostly combinational. It has no internal state — it just wires decoders to rename to RS to ROB. The state lives in the modules it connects.

6.5 Notes on the Widened RS

The Week 4 RS module had one dispatch port. For Week 5 integration, you need to widen it to two dispatch ports (slot A and slot B). Each port writes one new RS entry per cycle. Use the same priority-encoder pattern: slot A's dispatch picks the lowest-index free RS, slot B picks the next-lowest. The Verilog change is small (duplicate the dispatch port logic with priority), but be careful that A and B don't both pick the same RS slot.

Concept Check 5

1. In `dispatch.v`, why is the back-pressure signal `fe_stall_o` computed combinatorially (no clock)?
2. In the widened `rename_unit.v`, suppose A and B both want to allocate, but `fl_count = 1`. What is `stall_A_o`? `stall_B_o`? Walk through what happens to A and B.
3. In `dispatch.v`, the slot B RS dispatch uses `rob_tail_i + (A_dispatches ? 1 : 0)`. Why the +1 if A dispatches?

7 The Integration Test [~1 hour]

By the end of this week, the front-end (fetch → decode → rename → dispatch) must run a real program. The test verifies that the modules connect correctly and that intra-bundle bypass works.

7.1 The Test Program

Create `programs/test1.hex` with 10 RV32I instructions. Suggested:

```
# Test program: exercises intra-bundle bypass and WAW within bundle

# Bundle 1: independent
addi x1, x0, 10      # x1 = 10
addi x2, x0, 20      # x2 = 20

# Bundle 2: B depends on A (intra-bundle RAW)
add  x3, x1, x2      # x3 = x1 + x2
add  x4, x3, x1      # x4 = x3 + x1    (depends on previous add!)

# Bundle 3: WAW within bundle
add  x5, x1, x2      # x5 = x1 + x2
add  x5, x3, x4      # x5 OVERWRITES previous (WAW within bundle)

# Bundle 4: B reads what A reads (no dependence)
add  x6, x1, x2      # x6 = x1 + x2
add  x7, x1, x3      # x7 = x1 + x3 (both read x1, independently)

# Bundle 5: B writes what A reads (no dependence within bundle, but A still
add  x8, x6, x7      # x8 = x6 + x7
add  x6, x1, x1      # x6 = x1+x1 (later x6 write)
```

You'll need to assemble this by hand (or use a small Python script) into a hex file. Each line in the hex file is one 32-bit instruction in hex. The first few lines:

```
00A00093    # addi x1, x0, 10
01400113    # addi x2, x0, 20
00208193    # add  x3, x1, x2
... (compute the rest, or use the Python helper in the appendix)
```

7.2 The Testbench Structure

`tb_frontend.v` instantiates:

- One fetch module.
- Two decode modules (one for slot A, one for slot B).

- One (widened) `rename_unit` module.
- One `dispatch` module.
- One `reservation_station` module (widened to 2 dispatch ports).
- Stubs for ROB tail and commit (just counters).
- A fake PRF ready-bit vector (so RS wakeup doesn't fire, since we don't have execution yet).

The testbench:

1. Loads `test1.hex` into the imem via `$readmemh`.
2. Releases reset.
3. Runs the clock for ~20 cycles.
4. At every cycle, prints which instructions dispatch and which RS slot they occupy.
5. At the end, prints the final rename map state.

7.3 What to Verify Visually in the Waveform

Take a screenshot showing:

- Cycles 3, 4: both Bundle 1 instructions (`addi x1,...` and `addi x2,...`) dispatch in the same cycle. Two RS entries get written.
- Cycle 5: Bundle 2's `add x4, x3, x1` (slot B) has `rs_disp_B_pj` equal to the freshly-allocated P32 (or wherever A's destination went), *not* the stale rename map entry for x3.
- Cycle 7: Bundle 3's two `add x5` instructions both dispatch; `rename_map[5]` at the end of cycle 7 equals B's destination, not A's.

7.4 Expected Final State

After all 10 instructions dispatch (5 cycles, 2 per cycle, ignoring stalls):

- `rename_map[1]` → P32 (or whichever was first-allocated)
- `rename_map[2]` → P33
- `rename_map[3]` → P34
- `rename_map[4]` → P35 (uses P34 as source via bypass)

- `rename_map[5]` → P37 (the second `add x5` wins; P36 was the first, freed when slot B's `old_phys_dest` captures it)
- `rename_map[6]` → P41 (the second `add x6` in bundle 5 wins; first was P38, captured as `old_phys_dest`)
- `rename_map[7]` → P39
- `rename_map[8]` → P40

(Exact physical register numbers depend on free list state. What matters is the *pattern*: WAW is resolved, intra-bundle RAW uses the freshly-allocated phys reg.)

Concept Check 6

1. In Bundle 2 (`add x3, x1, x2 / add x4, x3, x1`), what value does the bypass logic select for slot B's `rs1_phys`? Where does it come from?
2. In Bundle 3 (two `add x5s`), suppose A is allocated P36 and B is allocated P37. What is slot B's `old_phys_dest`?
3. Why does the testbench use a fake PRF ready-bit vector instead of instantiating the actual `register_file` module?

8 Week 5 Deliverables

In Your Repo

- `rtl/fetch.v` — 2-wide fetch.
- `rtl/decode.v` — single decoder (instantiated twice).
- `rtl/rename_unit.v` — widened 2-wide rename with intra-bundle bypass (**replaces** the Week 3 version).
- `rtl/dispatch.v` — 2-wide dispatch.
- `rtl/reservation_station.v` — widened with 2 dispatch ports (**replaces** the Week 4 version).
- `tb/tb_frontend.v` — integration testbench.
- `programs/test1.hex` — 10-instruction RV32I test program.
- `sim/waveforms/frontend_dispatch.png` — waveform screenshot.
- Updated `learning_log.md`.

What Mentor Will Check

1. **Compiles in Quartus** without errors or critical warnings.
2. **Integration test runs to completion** with two-instructions-per-cycle dispatch in the common case (`tb_frontend.v` prints expected output).
3. **Intra-bundle RAW bypass works:** Bundle 2's slot B uses A's fresh phys dest as its source. Easy to verify in the waveform.
4. **Intra-bundle WAW resolved:** after Bundle 3, `rename_map[5]` = B's destination.
5. **Stall test:** when free list runs low, dispatch stalls correctly. (Not strictly required to test this week, but the logic should be present.)
6. **Plain Verilog only.** No SystemVerilog.

Common Bugs to Watch For

- **Bypass logic missing or wrong** on slot B's sources. The symptom is silent wrong values in the RS source fields.
- **Free list head advances by wrong amount.** Should be +2 if both allocate, +1 if only one, 0 if neither.

- **Same-cycle WAW not handled in rename map update.** If you blindly write both A's and B's mappings, the simulator may produce nondeterministic results. Use the priority pattern: B's write wins.
- **Back-pressure not propagated to fetch.** If dispatch stalls but fetch advances, you lose instructions.
- **Dispatch slot B's RS index aliases slot A's.** Make sure the priority encoder in the RS picks distinct slots for A and B.

Time Estimate

- Theory (Sections 1–4): ~2 hours.
- Verilog reading (Section 5): ~1 hour.
- Widening rename_unit.v: ~2 hours (this is the trickiest part).
- Writing fetch.v + decode.v + dispatch.v: ~3 hours.
- Widening reservation_station.v: ~1 hour.
- Integration test + debugging: ~1–2 hours.

A Appendix: Solutions to Concept Checks

Concept Check 1 (Going Wide)

1. Ideal: $IPC = 4$. On typical code: $IPC \sim 2\text{--}3$ due to dependencies. Infinitely wide: limited by data dependencies and branch mispredicts; theoretical maximum is the “critical path” length of the program divided by total work, often called the “critical-path IPC.” Real workloads typically max out at $IPC \sim 5\text{--}6$ even on extremely wide machines.
2. $6 \text{ reads} = 2 \text{ instructions} \times (2 \text{ sources} + 1 \text{ destination-old-mapping read})$. Each dispatched instruction needs to read its sources (for the RS) and the old mapping of its destination (for the ROB `old_phys_dest` field). $2 \times 3 = 6$.
3. Example: `addi x5, x0, 10 / addi x6, x0, 20`. Both instructions are I-type with `x0` as source (always reads zero), different destinations, no dependence, no shared writes.

Concept Check 2 (Fetch and Decode)

1. Because the immediate is a signed 12-bit value. The ALU operates on 32-bit values. Without sign extension, a negative immediate (e.g., -5, encoded as `0xFFB`) would be interpreted as a large positive value (4091), giving wrong arithmetic. Sign extension preserves the meaning.
2. It should produce a NOP or an exception flag. PRAVAH’s decoder defaults to `op = 0` (ADD), which is harmless but technically incorrect. For correctness, mark the decoded instruction as an unsupported-opcode exception in its ROB entry, which the ROB will trap on at commit.
3. Decode happens combinatorially in the cycle after fetch. The pipeline-register boundary between decode and dispatch is at the end of the same clock cycle as decode — decode reads the latched fetch outputs at the start of the cycle, computes micro-ops, and dispatch consumes them combinatorially before the next clock edge. (In some designs, there’s an explicit decode-to-dispatch pipeline register; PRAVAH combines them.)

Concept Check 3 (Intra-Bundle Dependencies)

1. `I4’s old_phys_dest = P32` means: when `I4` commits, `P32` (which was `I1`’s destination, briefly the architectural value of `x5`) is freed back to the free list. This is correct: by the time `I4` commits, `x5`’s architectural value is whatever `I4` produced (`P35`), so `P32` is no longer architecturally needed. `P32` has been “displaced” by `I4`’s new mapping.

2. `AB_write_same = 1`. B's `old_phys_dest` = A's freshly-allocated phys register (let's say P32). This is correct because just before B's dispatch, the rename map said `x5` → P5 (the original), but A's update would make it `x5` → P32. From B's perspective, the "current" mapping it's displacing is A's, not the original. So when B commits, freeing P32 is right — P32 was the architectural value of `x5` between A and B's commits.

Why doesn't A's update leak: the rename map is a clocked register; A's update isn't visible until the next cycle. The bypass logic does the work explicitly.

3. B's effects must be undone. The ROB walks from tail back to the branch (A), restores rename map entries from `old_phys_dest`, returns new phys registers to the free list. The RS entry for B (if it dispatched) is invalidated — we'd need a signal from the ROB to the RS array to clear specific slots. Week 6 adds this.

Concept Check 4 (Dispatch and ROB Interface)

1. Fetch must hold the bundle. The next cycle's bundle still has B as the lead, plus a new instruction (call it C) fetched alongside. Effectively, fetch advances PC by 4 (one instruction), not 8. This is the half-dispatch case; PRAVAH handles it by stalling fetch entirely until both slots can dispatch (the conservative policy).
2. `phys_dest` is what the ROB will use to find the result at commit (read `PRF[phys_dest]`, write to `arch_dest`). `old_phys_dest` is what gets freed (returned to the free list) when this instruction commits. Both are needed: commit frees the old, makes the new architectural.
3. Yes, both can dispatch in the same cycle as long as 2 RS slots are free. The RS module's 2-port dispatch picks distinct slots via priority encoder.

Concept Check 5 (Verilog)

1. Because back-pressure must propagate *in the same cycle* that dispatch decides to stall. If fetch advanced and *then* got the stall signal, it would already be too late — the dropped instruction is lost. Combinational paths from dispatch to fetch make the stall effective immediately.
2. `stall_A_o = 0` (free list has 1 entry, enough for A alone). `stall_B_o = 1` (need 2 entries for both). Result: A dispatches with P32; B does not dispatch this cycle. Fetch holds; B will retry next cycle when the free list has recovered (via commits).
3. Because A occupied `rob_tail` and B will occupy `rob_tail+1`. The ROB needs a unique index per dispatched instruction. If A dispatches, B's ROB index is one higher.

Concept Check 6 (Integration Test)

1. Slot B's `rs1_phys` = A's freshly-allocated `phys` dest (the one for `x3`). This comes from the bypass logic in `rename_unit`: `B_src1_uses_A` is true because B reads `x3` and A writes `x3`.
2. Slot B's `old_phys_dest` = P36 (A's destination). `AB_write_same` is true, so B's `old_phys_dest` captures A's freshly-allocated `phys`.
3. Because we don't have execution this week. The PRF ready bits won't be set by an FU writeback, so the RS would never see operands "become ready" and would never issue. The fake ready-bit vector lets us pretend all PRF entries are ready, but this isn't testing execution — we're testing dispatch, not the full pipeline. (In Week 6, the full register file is instantiated and the FU writes drive ready bits naturally.)

B Appendix B: Python Helper to Generate test1.hex

```
# generate_test1.py
# Run: python3 generate_test1.py > programs/test1.hex
# Produces the 10-instruction test program as a hex file for $readmemh.

def encode_r_type(func7, rs2, rs1, funct3, rd, opcode):
    return (func7 << 25) | (rs2 << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode

def encode_i_type(imm, rs1, funct3, rd, opcode):
    return ((imm & 0xFFF) << 20) | (rs1 << 15) | (funct3 << 12) | (rd << 7) | opcode

def addi(rd, rs1, imm):
    return encode_i_type(imm, rs1, 0b000, rd, 0b0010011)

def add(rd, rs1, rs2):
    return encode_r_type(0b0000000, rs2, rs1, 0b000, rd, 0b0110011)

instructions = [
    addi(1, 0, 10),      # addi x1, x0, 10
    addi(2, 0, 20),      # addi x2, x0, 20
    add(3, 1, 2),         # add  x3, x1, x2
    add(4, 3, 1),         # add  x4, x3, x1  (intra-bundle RAW)
    add(5, 1, 2),         # add  x5, x1, x2
    add(5, 3, 4),         # add  x5, x3, x4  (intra-bundle WAW)
    add(6, 1, 2),         # add  x6, x1, x2
    add(7, 1, 3),         # add  x7, x1, x3
    add(8, 6, 7),         # add  x8, x6, x7
    add(6, 1, 1),         # add  x6, x1, x1  (later x6 write)
]

for ins in instructions:
    print(f"{ins:08X}")
```